

Group Topic: Predicting Student Dropout Risk Using Academic and Behavioral Data **Lab No.**

Git Repo/ Colab Link:

Date: 5/16/2026

<https://github.com/dgbb-prog/student-dropout-bigdata-datacleaning>

Objectives

The purpose of this lab is to implement and evaluate classification algorithms for predicting student dropout risk. My personal objective was to implement three baseline classification models – Logistic Regression, Random Forest, and XGBoost – and record their accuracy on the test dataset. Logistic Regression is a simple linear model that predicts probabilities using a sigmoid function, making it fast and interpretable. Random Forest is an ensemble of decision trees that averages predictions to reduce overfitting and capture non-linear patterns. XGBoost is a boosting algorithm that builds trees sequentially, each correcting errors from the previous one, and often achieves the highest accuracy among the three. Together, these models provide a good baseline for comparing performance before fine-tuning.

Tasks Completed

The following baseline models were implemented this week to establish a performance benchmark for predicting student dropout risk. Each model represents a different approach to classification, from simple linear to complex ensemble methods.

1. **Received the preprocessed dataset:** My groupmates performed feature selection, encoding, scaling, and train-test split.
2. **Imported the required libraries** – I loaded LogisticRegression from scikit-learn for the linear baseline, RandomForestClassifier for the bagging ensemble, and XGBClassifier from the xgboost library for the boosting approach.

Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

3. **Initialized the three models** – I set random_state=42 for all models to ensure reproducible results, added max_iter=1000 to the Logistic Regression to guarantee convergence, and used eval_metric='logloss' for XGBoost to define the loss function for binary classification.

Code:

```
models = {  
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),  
    'Random Forest': RandomForestClassifier(random_state=42),  
    'XGBoost': XGBClassifier(random_state=42, eval_metric='logloss')  
}
```

- 4. Trained each model** – I iterated through the dictionary of models and called the .fit() method on each one using the training data (X_train, y_train) to learn the underlying patterns associated with student dropout.

Code:

```
for name, model in models.items():  
    model.fit(X_train, y_train)
```

- 5. Made predictions and calculated accuracy** – Using .predict() on the test set (X_test) for each trained model, I obtained predicted labels and then computed the baseline accuracy by comparing these predictions with the true labels (y_test) using accuracy_score().
- 6. Calculated and recorded baseline accuracy** – Computed accuracy using accuracy_score() and stored results.

Code:

```
from sklearn.metrics import accuracy_score  
for name, model in models.items():  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
    print(f'{name} Accuracy: {acc:.4f}')  
  
results = {}  
for name, model in models.items():  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
    results[name] = acc  
    print(f'{name} Accuracy: {acc:.4f}')
```

Results and Analysis

This section presents the baseline accuracy results from the three classification models I implemented. Below are the accuracy scores I obtained after training each model on the training set and evaluating on the test set.

Model	Baseline Accuracy
Logistic Regression	81.99%
Random Forest	84.06%
XGBoost	84.32%

```
*** Logistic Regression Accuracy: 0.8199
Random Forest Accuracy: 0.8406
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [05:06:22] WARNING: /_w/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

    bst.update(dtrain, iteration=i, fobj=obj)
XGBoost Accuracy: 0.8432
```

Key Findings:

- **XGBoost** achieved the highest baseline accuracy at 84.32%, followed closely by Random Forest at 84.06%. Both ensemble methods outperformed Logistic Regression by approximately 2 percentage points.
- **Logistic Regression**, despite being the simplest model, still achieved a respectable 81.99% accuracy, confirming that linear relationships exist between the features and dropout status.
- **Random Forest** demonstrated strong performance (84.06%) despite not being fine-tuned, proving that bagging with multiple decision trees is already highly effective for this dataset even with default hyperparameters.

The close performance between Random Forest and XGBoost suggests that both bagging and boosting ensemble methods are well-suited for this dropout prediction task. These baseline results provide a solid reference point for my groupmates to proceed with model evaluation (confusion matrices, ROC-AUC) and fine-tuning using GridSearchCV.

Challenges and Solutions

Challenge	Solution
XGBoost UserWarning about use_label_encoder being deprecated	The parameter was kept because it does not affect model accuracy; the warning is harmless and can be ignored.
Ensuring reproducibility – models gave slightly different accuracies each run	Set random_state=42 for all models to fix random seeds.
Logistic Regression did not converge after default iterations	Increased max_iter to 1000 to allow more iterations for convergence.

Conclusion

I successfully implemented three baseline classification models (Logistic Regression, Random Forest, and XGBoost) for predicting student dropout risk. XGBoost achieved the highest accuracy at 84.32%, establishing a strong baseline for my groupmates to improve upon through fine-tuning and hyperparameter optimization. I learned how to properly initialize and train different classification models in Python, the importance of setting random states for reproducibility, and how to handle common warnings like XGBoost's deprecated parameters. My work provides the foundation for the group's next steps in model evaluation and fine-tuning.